

PROJECT REPORT

PneumoFL

Privacy-Preserving Pneumonia Detection via Federated Learning

What it is, how it was built, and what went wrong along the way

Course	B.Tech BCSE497J — Project I, SCOPE
Team	Uddhav Sethi (23BKT0092), Chirag Gadhyan (23BDS0265), Ishaan S Shrivastav (23BCI0130)
Faculty guide	Dr. Adaline Suji
Alignment	UN Sustainable Development Goal 3 — Good Health & Well-Being
Repository	UddhavSethi/privacy-preserving-medical-diagnosis
Report date	2 September 2026 (updated — original 31 August 2026)

1 What This Project Is, and What's Actually New About It

PneumoFL trains a chest X-ray pneumonia classifier across three simulated hospitals that never share a single patient image with each other or with a central server. Instead, each hospital trains locally and sends only a small, noised, cryptographically masked update of model weights — the images themselves never leave the hospital's own storage.

1.1 The problem it addresses

Three separate, well-known problems compound in medical imaging AI, and most published work only tackles one at a time:

- **Data cannot be centralized.** Pooling patient X-rays from multiple hospitals onto one server for training is usually illegal, unethical, or contractually impossible. Most academic pneumonia-detection papers sidestep this by simply using an already-pooled dataset.
- **Federated updates are not automatically safe.** Even when raw images stay local, the model updates a hospital sends can still leak information about its training data (membership inference, gradient inversion). Treating "federated" as synonymous with "private" is a common but incorrect assumption.
- **Black-box predictions aren't clinically actionable.** A prediction with no explanation and no confidence signal gives a clinician nothing to act on or challenge.

1.2 The system

PneumoFL layers six mechanisms on top of a DenseNet121 classifier, each protecting or explaining a different part of the pipeline:

Layer	Protects / provides	Mechanism
Federated Learning	Raw patient images stay local	Flower — hospitals train locally, server only aggregates
Differential Privacy	Information inside an update	Opacus DP-SGD — per-sample clipping + calibrated Gaussian noise, formal (ϵ, δ) accounting
Secure Aggregation	An individual hospital's update, even from the server itself	Flower SecAgg+ — cryptographic masking, server only ever recovers the sum
TLS + client authentication	Messages in transit; impersonation resistance	gRPC + mutual TLS, per-hospital certificates
Explainability	Clinical trust in individual predictions	Grad-CAM heatmaps over the model's final conv layer
Uncertainty estimation	Deferral of low-confidence cases to a human	Monte Carlo Dropout, fixed-coverage deferral policy, plus a calibrated decision threshold and a genuine "Uncertain" abstention state

Added in this update: a seventh, standalone mechanism — a supervised chest-X-ray input gate (§3 Part D) that rejects non-medical images before any of the six above ever run on them, on top of everything already listed.

1.3 What's actually novel here — checked against real published work, not assumed

It would be easy to claim "nobody has combined all of these before" without checking. That claim was tested against the current literature rather than taken on faith. The closest published comparator remains **FedIHRAS** (Kollias et al., *Biomedicines*, 2026) — a federated radiological-analysis framework that also combines federated learning, differential privacy, secure aggregation, and Grad-CAM explainability. It is a real, close precedent, and should be named as one rather than talked around. It differs from PneumoFL in scope: FedIHRAS uses a ResNet-50 backbone and adds anatomical segmentation and automated SNOMED-CT report generation, but — based on its published description — does not appear to include TLS-level client authentication as a named component, a Monte Carlo Dropout uncertainty/deferral layer, or a systematic ablation quantifying what each privacy layer individually costs.

This update re-ran the literature check rather than reusing the original one uncritically. Two additional pneumonia-specific frameworks surfaced — **FLPneXAINet** (Biswas et al., *PLOS ONE*, 2025) and **U-FDL-PPE** (2025) — both combining federated learning with Grad-CAM/LIME explainability for chest X-ray classification. Neither includes differential privacy, secure aggregation, or any uncertainty-quantification mechanism; both stop at "federated + explainable," not "federated + private + secure + explainable + uncertainty-aware." Their existence corroborates rather than undermines the original claim: several papers now combine *some* subset of these mechanisms for chest X-ray work specifically, but none found combine all of them with a systematic per-layer cost ablation.

The fair, literature-grounded claim is therefore narrower than "first ever" and more specific: **the particular six-layer combination (FL + DP + SecAgg + TLS/client-auth + Grad-CAM + MC-Dropout deferral), implemented together as one running pipeline, with an explicit ablation ladder quantifying the accuracy/calibration/communication cost each layer adds on top of the others**, is uncommon in the literature surveyed. Individual pairs of these mechanisms are well studied; assembling and *measuring* all six together, rather than asserting they compose safely, is the part this project actually contributes.

The most persuasive single number the project produces is the federated model outperforming any individual hospital's locally-trained model — the actual value proposition of federated learning, measured directly rather than assumed — alongside a full epsilon sweep showing the privacy/accuracy tradeoff, a quantified finding that the frozen-backbone architecture required for that privacy stack measurably degrades Grad-CAM explanation quality, and confirmation that unfreezing and fine-tuning the last block under real federated averaging recovers a meaningful share of that lost explanation quality and accuracy — at a real, honestly-reported cost: a 24% missed-pneumonia rate at a naive 0.5 decision threshold, since improved by deriving an operating threshold from real validation data, and a further finding (added in this update) that nearly all of the remaining false-positive gap traces to RSNA's own label semantics (0.856 AUROC on the realistic full population vs. 0.970 restricted to unambiguous cases), not a fixable model weakness.

Sources consulted: FedIHRAS — Kollias et al., PMC13023735 / doi:10.3390/biomedicines14030713; DP+FL COVID chest X-ray, PMC11193384; federated pediatric pneumonia classification, Scientific Reports (2024); privacy-preserving FL + uncertainty quantification review, Radiology: AI and arXiv:2409.16340; FL+DP medical imaging with parameter ablations, Scientific Reports (2022); FLPneXAINet — Biswas, Mostafiz, Uddin & Uddin, PLOS ONE (2025), PMC12270172; U-FDL-PPE, PMC12713205; Zulqarnain et al., Healthcare Technology Letters (2026).

2 Technology Stack, and Why Each Piece Was Chosen

Every dependency below was picked for a specific reason tied to the project's priority order — privacy, then security, then correctness, then reproducibility, then academic credibility, then explainability, then uncertainty, then maintainability, then raw accuracy — never the reverse.

Layer	Technology	Why this, specifically
Language	Python 3.11 (pinned, not 3.12)	Exact version compatibility across the whole FL/DP toolchain (Flower, Opacus) — a mismatch here is a known source of silent breakage.
Model backbone	DenseNet121, ImageNet- pretrained	CheXNet lineage — the standard, reviewer-familiar choice for chest X-ray work. Not swapped for anything newer/flashier because reviewer familiarity and comparability mattered more than novelty here.
FL framework	Flower (flwr)	The only mainstream framework offering both a simulation engine (fast sweeps) and a real deployment engine (gRPC, real processes) behind the <i>same</i> client/server code — critical for the project's "simulate for measurement, deploy for demonstration" split.
Aggregation strategy	FedAvg	The standard, best-understood baseline. FedProx/FedBN were deliberately not substituted in — introducing a second algorithmic variable would have muddled the ablation's causal story.
Differential Privacy	Opacus (DP-SGD)	The PyTorch-native library with a real formal accountant (RDP), not just noise injection — a privacy claim without an accountant is explicitly treated as unacceptable in this project.
Secure Aggregation	Flower SecAgg+	An established, peer-reviewed masking protocol. Writing custom cryptography was explicitly ruled out — a hand-rolled scheme is a near-guaranteed reviewer attack surface.
Transport security	gRPC + mutual TLS	Server-only TLS doesn't stop a fake hospital from connecting; client authentication was required to actually back the project's impersonation-resistance claim.
Preprocessing (contrast)	OpenCV (CLAHE only)	torchvision has no CLAHE implementation; used narrowly for exactly that one operation, with parameters fixed, logged, and precomputed to disk rather than left as a per-run source of drift.
Transforms/augmentation	torchvision	Correct, composable seeding for augmentation — kept separate from CLAHE specifically so random augmentation state is never entangled with a deterministic preprocessing step.
Explainability	pytorch-grad-cam	An established library instead of a hand-rolled Grad-CAM implementation, which also made comparing Grad-CAM

variants close to free.

Uncertainty	Monte Carlo Dropout	Cheap, and the method the source project deck specified. Explicitly framed as a known-weak baseline, not the strongest possible choice — stronger methods are named as future work rather than silently substituted in.
Config management	Hydra / OmegaConf	Every run fully specified by config, no hand-edited constants between runs — a reproducibility requirement, not a convenience.
Experiment tracking	MLflow (local, self-hosted)	Chosen deliberately over a cloud tracking service — a project whose whole thesis is data sovereignty should not ship its own telemetry to a third party.
Containerization	Docker Compose	Makes the multi-hospital deployment demonstration reproducible off the original machine.
Dataset acquisition	kaggle, pydicom, Pillow	RSNA is DICOM (needs pydicom); Kermany is JPEG (needs Pillow). Kaggle's API automates checksum-verified acquisition of both.
Metrics	scikit-learn	AUROC as the primary metric (not raw accuracy) — mandatory on this imbalanced medical dataset. Also backs this update's new X-ray gate (logistic regression) — no new dependency needed.
Demo interface	Streamlit	The one dependency approved specifically for a presentation-only layer over already-trained checkpoints — deliberately does not touch training, evaluation, or the FL pipeline.

Deliberately excluded

TensorFlow / TF-Federated / TF-Privacy, NVFlare, OpenFL, PySyft, CrypTen, Weights & Biases or any cloud tracking service, blockchain components, Vision Transformers or medical foundation models, and Kubernetes were all considered and explicitly ruled out — several for the same reason (a second, redundant framework for something Flower/Opacus already does), and cloud tracking specifically because it would contradict the project's own data-sovereignty premise.

No new external dependency was added in this update's inference-pipeline or deployment-hardening work — the decision-threshold, calibration, abstention, and X-ray-gate logic (§3 Part D) are all built entirely on PyTorch and scikit-learn, already pinned.

3 Everything That Went Wrong, and How It Was Fixed

This project's own working culture treats "found via a real run, not by inspection" as worth stating explicitly wherever it applies — a recurring pattern below. Issues are grouped by when they surfaced: the original 24-stage build (Stages 0-23 and the OPT-1-5 research extensions), an earlier debugging session (Part B), an update's own initial work (Part C), and this same update's continuation, including a pre-deployment hardening pass (Part D).

PART A — DURING THE ORIGINAL BUILD (STAGES 0-23, OPT-1 THROUGH OPT-5)

STAGE 5 RSNA validation/test shard collapsed to one hospital

What went wrong: The RSNA shard-splitting step reused the exact same seed (1000) as Stage 4's earlier patient-level train/val/test split on the same population. Because both used a sort-then-shuffle pattern, the identical seed reproduced the identical cut points instead of an independent split — Hospital C ended up with 100% train data and zero validation/test records; Hospital B absorbed all of it instead.

Fix: Changed the shard seed to 5000, regenerated both partition files, and added a regression test so the two seeds can never silently collide again.

STAGE 8 In-place ReLU silently broke per-sample gradients

What went wrong: The classifier head used `nn.ReLU(inplace=True)`, which conflicts with Opacus's backward hooks — in-place ops break the view-tracking `GradSampleModule` needs to compute per-sample gradients for DP-SGD. Found by actually running Opacus's validator, not by reading the docs.

Fix: Switched to `inplace=False`.

STAGE 13 Six real Flower-tooling gaps, found only by running it

What went wrong: The pinned Flower version had fully moved to a new Message API, superseding the older client-function style most examples online still show. On top of that: the local simulation needs a persistent SuperLink daemon that can accumulate stale state across killed runs; the default simulated node count is 2, not 10; the packaging step's default include patterns swept in ~350MB of unrelated files, blowing a 10MB bundle limit; the fix for that (a positive allowlist) silently no-ops if nested under the wrong config table, with no error; and the simulation runtime executes from a copied, isolated directory, so any path derived from `__file__` silently resolves wrong. Separately, the local training loop seeded only its shuffle order, leaving Dropout's own random mask unseeded — "identical" reproducible runs silently diverged.

Fix: Rewrote the client/server on the Message API; documented the daemon's known failure mode (kill it, delete its state DB, retry clean); pinned node count explicitly; switched to a positive file-allowlist threaded through config; moved every path to an absolute one threaded through config; added an explicit `torch.manual_seed(seed)` alongside the local shuffle generator.

STAGE 14 DP accountant mismatch and an overflow crash

What went wrong: The noise calibration function assumed an RDP accountant, but a bare `PrivacyEngine()` defaults to a different one ("prv") — the calibrated noise and the reported epsilon would have been computed under two different mathematical models, silently. Separately, `get_epsilon()` overflowed instead of returning infinity as noise approaches zero, crashing the no-DP-equivalent edge case. A third issue was methodological: an early test tried to verify DP clipping/noise by comparing Adam-optimized parameter positions before and after, which gave the wrong-direction result because Adam's adaptive normalization confounds a raw-gradient comparison.

Fix: Pinned the accountant explicitly to RDP everywhere; wrapped epsilon computation in a guard for the overflow case; rewrote the test to inspect Opacus's internal clip/noise mechanism directly instead of optimizer output.

STAGE 15 Secure Aggregation's default overflowed on real hospital sizes

What went wrong: Found live: Flower's SecAgg+ workflow defaults to `max_weight=1000.0`, far below this project's real per-hospital training counts (~13,342 for the RSNA shards) — every round silently logged an overflow warning in the weight-encoding math.

Fix: Set `max_weight=20000` explicitly, with headroom for future partitions.

STAGE 16 A rejected client hung the test suite indefinitely

What went wrong: A negative-path test (an unregistered node attempting to connect) hung forever — `flower-supernode` doesn't exit cleanly on an auth rejection because its sidecar process keeps stdout open, and a plain `subprocess.run(timeout=...)` can't reach a subprocess's own children.

Fix: Launched with `start_new_session=True` and killed the whole process group via `os.killpg` on timeout.

STAGE 17 Three separate Docker deployment bugs

What went wrong: (1) Excluding `torch` by name from the Docker build still pulled ~2GB of unrelated CUDA packages, since excluding one package doesn't prune its transitive dependencies from a CUDA-oriented lockfile. (2) A TLS certificate's SAN list didn't include the Compose service name `superlink`, so all three hospital containers hung in uninformative silent retry loops instead of a clear certificate error. (3) A registration race: all containers started simultaneously, but hospital registration happened afterward and sequentially, so a hospital's first connection could land before its key was registered — a hard crash with no retry.

Fix: Used direct pinned-CPU-version installs for the Docker build only; added the missing SAN entry; reordered startup to register every hospital before any hospital container starts.

STAGE 18 Grad-CAM hard-crashed on ADR-1's frozen backbone

What went wrong: `pytorch-grad-cam`'s plain `GradCAM` doesn't mark its input tensor as requiring gradients by default. With the backbone entirely frozen, nothing in the forward pass required grad at all, so no autograd graph was ever built and the backward hook silently never fired — a hard crash, caught on this stage's own first live run. A second, smaller bug: the qualitative-example script collected records source-first, so Kermanshah's small bucket filled the sample slice before any RSNA record was reached.

Fix: Explicit `requires_grad_(True)` on the input tensor before calling into the library; a seeded shuffle before bucketing examples.

STAGE 21 Four bugs in the full ablation campaign

What went wrong: (1) The feature-loading function assumed one data source per hospital, which would have silently dropped every record from the second source for Dirichlet's mixed-source synthetic clients. (2) Dirichlet partitioning's own client keys didn't match the client app's hardcoded hospital mapping. (3) No federated run had ever actually been logged to MLflow before this stage. (4) The campaign's actual first launch attempt corrupted itself within seconds: the runner script omitted Flower's `--stream` flag, so it submitted and returned immediately instead of blocking, and all 27 runs got dispatched to the same SuperLink almost simultaneously. A fifth issue: MLflow's SQLite filter parser rejects parenthesized boolean grouping, and naive substring name-matching would have silently pulled DP runs into the plain-FedAvg row.

Fix: Grouped features per (source, split) and concatenated; remapped Dirichlet client keys to match; added optional MLflow logging to both server apps; added the missing `--stream` flag and relaunched the campaign clean; switched to flat filter clauses on logged params instead of name pattern-matching.

STAGE 23 A stray test run contaminated the production results database

What went wrong: An integration smoke test invoked a real federated run but never redirected its MLflow tracking URI, so every test execution silently wrote a stray result into the actual production results database. Caught only because a regenerated chart showed `n_seeds: 6` instead of the expected 3.

Fix: Root-caused the three specific stray runs by timestamp and a distinctive AUROC value, deleted them directly, then fixed the actual source by redirecting the test's tracking URI to an isolated temp file.

OPT-3 A background job reported finished while still running

What went wrong: Combining a manual `nohup cmd &` with the harness's own background-execution flag caused a spurious early completion report — the shell returned immediately because of the redundant trailing `&`, while the real ~12-minute Grad-CAM evaluation job was still executing.

Fix: Caught by checking the actual process and output directory before trusting the completion signal, then waited it out with a proper blocking poll instead of double-backgrounding.

SESSION The live app called a real pneumonia X-ray "Normal" with high confidence

What went wrong: Traced the full inference path end to end. The real cause: the app's default model (FedAvg + DP at $\epsilon=4$) was severely biased toward predicting "Normal" specifically on Hospital A's data — 54.9% accuracy there, worse than the 71.9% majority-class baseline, even though its AUROC (0.825) showed the underlying features were still informative. DP-SGD's per-sample clipping and noise had miscalibrated the decision threshold, not destroyed the learned representation.

Fix: Switched the app's default model to FedAvg without DP (82.8% accuracy, 0.950 AUROC on the same set). No model or training code was touched — an app-layer default change only.

SESSION The out-of-distribution warning fired on almost every real X-ray

What went wrong: Each hospital's anomaly detector is calibrated to flag only ~5% of its own held-out data, but the app showed its caution banner if *any single* hospital's detector disagreed with an image — a genuine Kermany X-ray was flagged 9/10 times, a genuine RSNA X-ray 10/10 times.

Fix: Changed the logic from "any hospital disagrees" to "no hospital recognizes it."

SESSION Grad-CAM was measurably looking at the wrong part of the X-ray

What went wrong: Measured directly on 150 real RSNA pneumonia X-rays with radiologist-drawn bounding boxes: correct predictions pointed inside the true opacity only 18.6% of the time; wrong predictions, 0.0%. Root cause: the frozen-backbone architecture compresses each X-ray into a single 1,024-number summary before the classifier ever sees it, so no spatial layout survives.

Fix: Implemented the project's own pre-approved fallback — unfreezing the backbone's last dense block and swapping its BatchNorm for GroupNorm. A bounded pilot confirmed real gains: pooled test AUROC 0.9051 → 0.9256; Grad-CAM pointing-game accuracy roughly doubled when correct (23.1% → 37.5%) and recovered from 13.6% to 35.2% on cases still misclassified.

SESSION The federated re-run of that fix stalled repeatedly, for three different reasons

What went wrong: First, forcefully killing an earlier stuck process left Flower's local SuperLink daemon without its execution companion process, so it silently accepted new runs without ever dispatching them. Second, background training runs were mysteriously killed mid-flight by something outside this conversation's control. Third, a new evaluation step added for best-checkpoint selection stalled for nearly two hours — Flower's simulation engine reuses the same GPU-bound actor pool for server-side evaluation immediately after client training, and that reuse hangs on a second heavy CUDA workload.

Fix: Restarted the SuperLink daemon fully; fully detached later runs from the harness's process tracking (`setsid + nohup + disown`); restructured the evaluation step to do no GPU work at all inside Flower's managed context.

SESSION The checkpoint-selection logic would have silently kept a worse model

What went wrong: Caught by the user, not found independently: the first version of the federated fine-tuning re-run saved whichever round happened to run last, regardless of whether a middle round had actually scored better.

Fix: Every round's checkpoint is now scored against a held-out pooled validation set, and only the best-scoring round is kept. Verified with a direct test: a deliberately destroyed round (AUROC collapsed to 0.5) was confirmed *not* to overwrite an earlier, genuinely better checkpoint.

PART C — THIS UPDATE, INITIAL WORK (1-2 SEPTEMBER 2026)

SESSION A day of GPU training nearly got lost to a laptop battery

What went wrong: The 10-round federated pilot trained cleanly overnight (client-side val AUROC 0.834 → 0.882 across all 10 rounds), but the follow-up per-round scoring step was killed mid-run when the laptop's battery died. No training was lost: every round's checkpoint had already been backed up.

Fix: Relaunched the same combined scoring script, fully detached, against the backed-up checkpoints. Round 9 selected as best by pooled validation AUROC (0.8507).

SESSION 78% accuracy was hiding a real clinical problem

What went wrong: A full confusion-matrix breakdown showed what AUROC alone couldn't: at the default 0.5 threshold, **24% of real pneumonia cases were being missed** (360 false negatives out of 1,502), and precision was only 0.618.

Fix: This finding directly motivated the threshold/calibration/abstention work that follows.

SESSION Round 9's checkpoint used a different file format than every other checkpoint

What went wrong: The app's checkpoint loader assumed every checkpoint is a bare classifier state dict — true for every existing checkpoint, but round 9's fine-tuned checkpoint stores a different, prefixed subset because more of the network was actually trained. Loading it the old way would have raised a hard key-mismatch error on first use.

Fix: Added an explicit flag to the loader that branches to the correct format. Verified against 8 real, known-labeled X-rays end to end — 7 of 8 correct, no crash.

SESSION Deferral and OOD checks would have silently scored the wrong feature distribution

What went wrong: The app's uncertainty and anomaly checks both read a feature cache computed under the assumption of a fully frozen backbone — invalid for round 9's partially-unfrozen one.

Fix (at the time): Both checks explicitly disabled for this checkpoint, with a visible note. (Re-enabled properly in Part D, §19.)

SESSION A rounding bug made 99.96% confidence display as a literal "100%"

What went wrong: The confidence meter rounded to zero decimal places, so a genuinely high — but not absolute — confidence displayed as an unqualified "100%".

Fix: Switched to one decimal place with an explicit floor, so a value just under 100% can never round up to a false "100.0%".

SESSION A hardcoded 0.5 decision threshold, and no way for the app to say "uncertain"

What went wrong: Every prediction was forced into Normal/Pneumonia regardless of how close the underlying probability sat to an implicit, never-varied 0.5 cutoff.

Fix: Swept the decision threshold on the validation set only. Adopted 0.45 — the Youden's-J optimum — for round 9: sensitivity 0.747 → 0.825 (missed-pneumonia count down 31% relative). Added a genuine "Uncertain" third result state for predictions in a data-derived band (± 0.05) around the threshold, matching this project's own 10%-deferral convention.

SESSION Calibrating for "overconfidence" found the opposite problem

What went wrong: The working assumption was that near-1.0 confidence must mean overconfidence. Fitting temperature scaling on round 9's real validation set found the opposite: the best-fitting temperature was 0.935 — *sharpening*, not softening.

Fix: The honest reading, reported rather than smoothed over: MC Dropout's own averaging already calibrates this checkpoint reasonably well; the true source of the "100%" complaint was the rounding bug above. Temperature scaling was still applied since the small improvement it measured was real.

SESSION The same SuperLink daemon leak recurred, twice, during this update's own test verification

What went wrong: The project's integration smoke test failed twice — not from anything in this update's own code, but from an orphaned SuperLink/Ray cluster from an earlier test invocation never getting torn down.

Fix (at the time): Manually killed the orphaned processes both times. (The actual root cause was found and fixed properly later — see Part D, §11.)

SESSION A process-hygiene mistake, caught and reverted within the same turn

What went wrong: While isolating a failing test, `git stash` was used without a specific file scope — reverting every uncommitted change in the repository, not just the files being tested.

Fix: Caught immediately and reversed with `git stash pop` in the same turn, before any further action. Verified every file was back exactly as it had been.

PART D — THIS UPDATE, CONTINUED: PRE-DEPLOYMENT HARDENING (2 SEPTEMBER 2026)

SESSION A non-X-ray upload still got a confident "Normal" diagnosis

What went wrong: A real photo (not a chest X-ray) returned a confident "Normal" result with a large headline and percentage confidence. The OOD detectors correctly flagged it — verified directly — but the app rendered the confident label unconditionally and appended the caution note as the very last item, literally after a "falls within the model's normal confidence range" reassurance for the same image.

Fix: An OOD-flagged image now shows "Cannot analyze" instead of a Normal/Pneumonia headline, with no confidence number and no Grad-CAM, and the caution message leads immediately.

SESSION That fix was itself too aggressive: it blocked a real X-ray, plus a real reproducibility bug

What went wrong (bug 1): The same uploaded image gave a different confidence value on repeated analysis — MC Dropout's stochastic passes were never seeded.

Fix: Seed from a hash of the uploaded image's own bytes. Verified: three runs of the same file now produce bit-identical confidence.

What went wrong (bug 2): A real chest X-ray downloaded from the web got no result at all — it tripped the exact same OOD flag a non-medical photo does, and the "Cannot analyze" hard block from the previous fix suppressed it entirely. The per-hospital detectors, calibrated on only two sources, can't reliably distinguish "not medical at all" from "a real X-ray processed differently than training data."

Fix: Reverted the hard block — the result is always shown again — while keeping the caution-note-leads improvement.

SESSION Building a real chest-X-ray input gate — two failed approaches, then one that worked

What went wrong / was tried: Owner asked for a real gate: X-rays allowed in any form, other images actually rejected. **Attempt 1** — mean color saturation: a real X-ray (0.072) and a non-X-ray logo wallpaper (0.079) were indistinguishable; a non-X-ray anime wallpaper (0.0008) was more grayscale than either. Rejected before writing code. **Attempt 2** — a supervised classifier trained on real X-rays vs. synthetic non-X-ray surrogates (random noise + geometric shapes, reusing OPT-5's own generator): perfect on held-out synthetic data, but misclassified two real held-out non-X-ray photos as X-rays (one at 92% confidence) — synthetic surrogates don't share real-photo feature statistics closely enough to generalize.

What worked: The same supervised setup with 35 real, locally-available photos (not a new dataset, nothing downloaded) mixed into the negative class. Validated against 6 different real photos held out entirely from training: **all 6 correctly rejected**, while both known real chest X-rays stayed correctly accepted. Only the fitted model's ~1,025 weights are committed — not the bootstrap photos themselves (copyright, and not portable to another machine). Wired in as a real pre-analysis gate using a dedicated always-frozen backbone instance, independent of whichever checkpoint is selected for diagnosis.

SESSION Extending threshold/calibration/abstention to the app's actual default checkpoint

What went wrong: All of the threshold-sweep/calibration/abstention work had been scoped to round 9 only — the checkpoint most people actually see (`fedavg_no_dp` , the real default) still had a naive, never-examined 0.5 threshold.

Fix: Ran the identical val-only methodology against it. Recommended threshold 0.45: sensitivity 0.787 → 0.841, missed-pneumonia count down 25% relative. Calibration result differed meaningfully from round 9's: fitted temperature 1.14 — genuine *softening* this time, since this checkpoint really is somewhat overconfident (unlike round 9). ECE improved ~46% relative, a much cleaner result than round 9's marginal one — two checkpoints, two different calibration directions, both measured rather than assumed.

SESSION The X-ray gate's negative set was almost entirely wallpapers

What went wrong: The gate validated well on 6 held-out real photos, but they were nearly all wallpapers — a narrow slice of "real photos" that doesn't say much about broader generalization.

Fix: Found 66 real photos across 6 visually distinct categories already on the local machine (terminal-theme screenshots, tool-documentation assets, a blockchain-assignment figure/logo set, security-tool screenshots). Held out 12 this time, including at least one from every new category: **12/12 correctly rejected** — stronger evidence of generalization, not just a bigger number.

SESSION Re-enabling deferral and OOD detection for round 9

What went wrong: Deferral/OOD had been disabled for round 9 (Part C) since Stage 9's feature cache assumes a frozen backbone. Built round 9 its own cache from its actual checkpoint — and hit a real bug immediately: `load_hospital_features` has a module-level cache-key constant baked into its lookup, so pointing it at a different directory alone wasn't enough; it still looked for the wrong hashed subdirectory.

Fix: Skipped Stage 9's 5 augmented training views entirely when building round 9's cache — neither deferral nor OOD ever reads them, cutting the build cost roughly 5x for zero functional difference (real run: under 16 minutes total). Added an optional, default-backward-compatible `feature_key` parameter rather than re-running the build to match an unrelated hardcoded assumption. Verified with real computed values: deferral threshold 0.6898, real per-hospital OOD thresholds — not placeholders.

SESSION Correction: the integration-smoke-test flakiness had a different, simpler cause

What went wrong: The recurring test failures (Part C) had been attributed to Flower's SuperLink daemon lifecycle — a real gap, but not, on direct investigation, what actually caused those specific failures. The real cause: `pyproject.toml`'s component-routing config was left pointed at the expensive fine-tuning app instead of the canonical one, because the battery-death crash killed the pilot script before its own cleanup could revert it — and that broken state had already been committed.

Fix: Reverted the config; a clean run now takes 36s instead of timing out at 180s. Two other real, independent robustness gaps found while chasing the misdiagnosis were fixed too (process-group and daemon-specific cleanup), though neither was the actual cause — recorded honestly rather than left conflated with the real fix.